

Kubernetes

Антон Титов

Руководитель группы защиты веб-приложений

Что мы уже знаем после Docker

Код

исходники приложения

Образ

шаблон запуска

Контейнер

изолированный процесс

Сеть

как контейнер общается

Volume

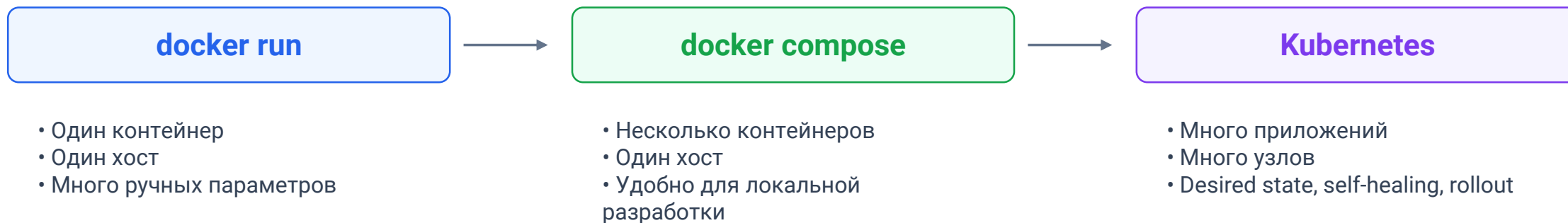
где хранить данные

Compose

несколько контейнеров на одном
хосте

Kubernetes начинается там, где ручное управление контейнерами становится ненадёжным: много хостов, много реплик, обновления, отказоустойчивость, сеть и доступы.

Docker run → Compose → Kubernetes



Compose полезен для понимания связки сервисов, сетей и volume. Но **Kubernetes** добавляет планировщик, контроллеры, кластерную сеть, API-объекты и отказоустойчивость.

Зачем нужен Kubernetes: реальные кейсы

Слишком МНОГО сервисов

10+ микросервисов сложно запускать вручную на Docker

Масштабирование

Сегодня 2 реплики, завтра 20 – без переписывания запуска.

Обновления без простоя

Нужно заменить v1 на v2 постепенно, а не «погасить всё».

Самовосстановление

Если процесс/узел умер – приложение должно вернуться само.

Секреты и конфиги

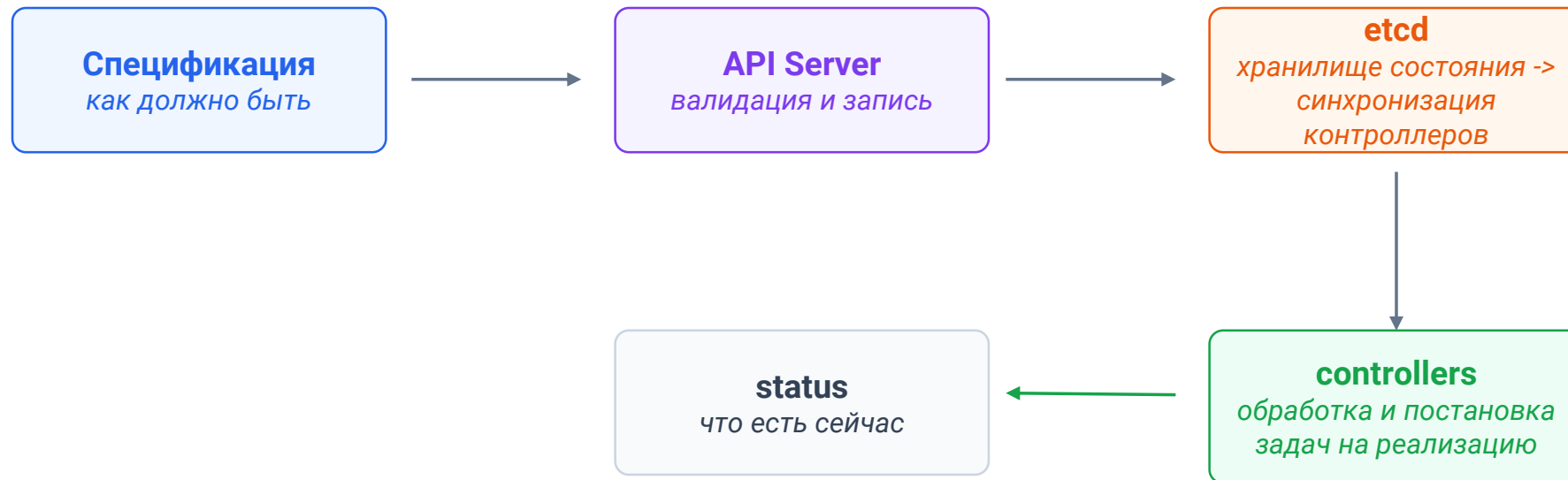
Один образ должен работать в dev/stage/prod.

Контроль ресурсов

Приложения бывают очень жадными. Их хотелки надо укротить

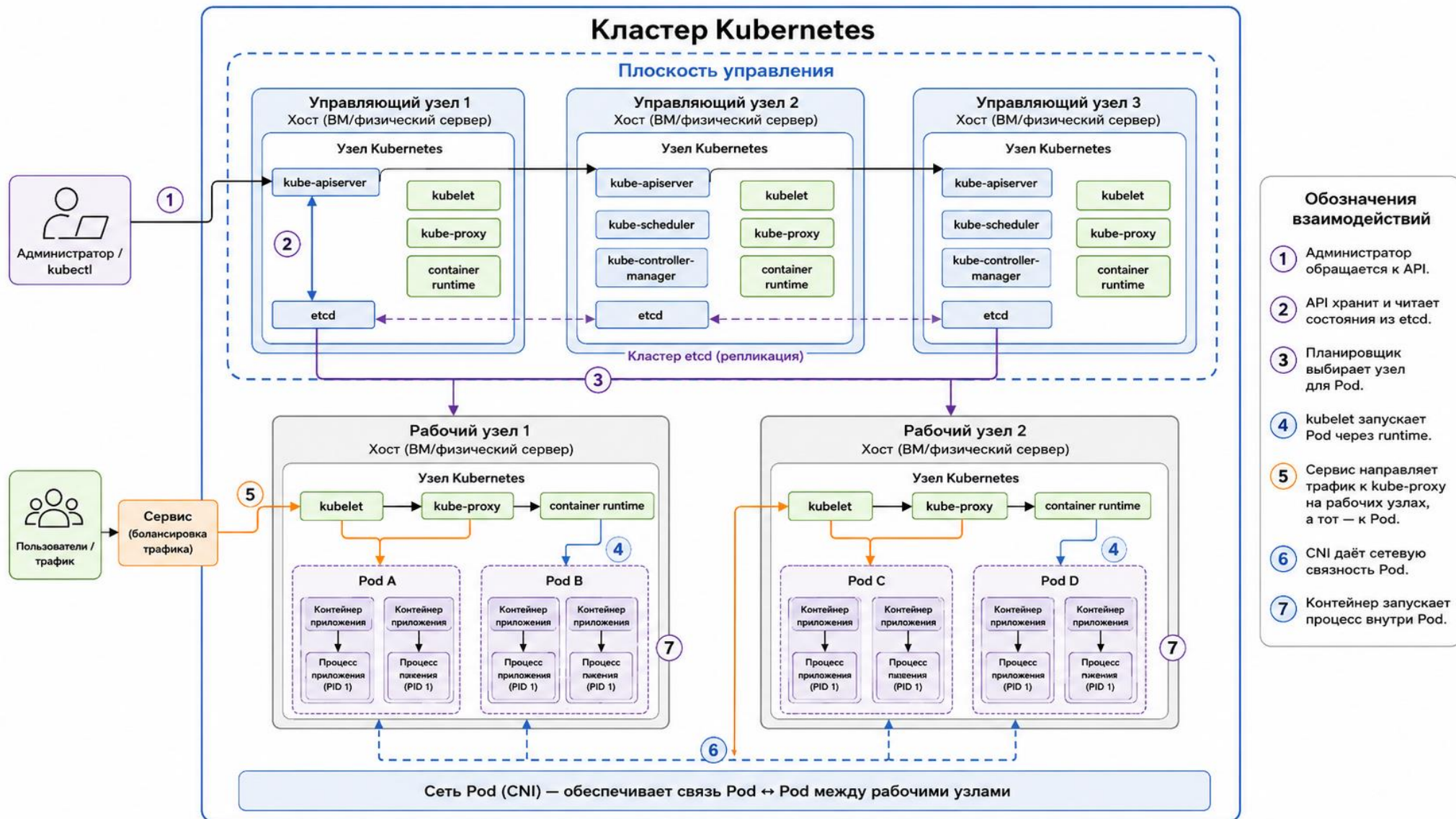
Вывод: Kubernetes – это про управление состоянием приложения в кластере, а не просто про запуск контейнера.

Главная идея Kubernetes: desired state



Контроллеры не «выполняют YAML». Они читают объекты API, сравнивают желаемое состояние с текущим и создают/изменяют другие объекты.

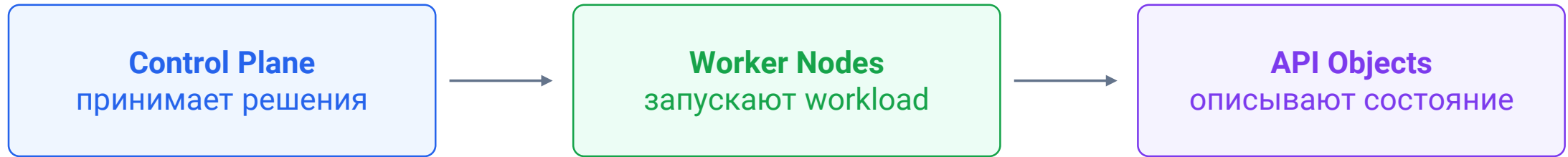
Общее архитектурное представление



Дальше разберём эту схему сверху вниз: control plane → worker node → Pod → сеть → storage → rollout → ИБ.

Что такое Kubernetes-кластер

Кластер – это набор хостов и компонентов, объединённых через Kubernetes API



В production control plane обычно делают отказоустойчивым: несколько управляющих узлов и кластер etcd.
Worker nodes могут добавляться и удаляться.

Важно: Kubernetes это не один демон, а набор процессов и API-объектов.

Плоскость управления: кто за что отвечает

Control Plane управляет кластером и хранит его состояние

kube-apiserver

Единая точка входа в Kubernetes API. Все изменения идут через него.

etcd

Распределённое key-value хранилище состояния API.

kube-scheduler

Выбирает узел для нового Pod с учётом ресурсов и правил.

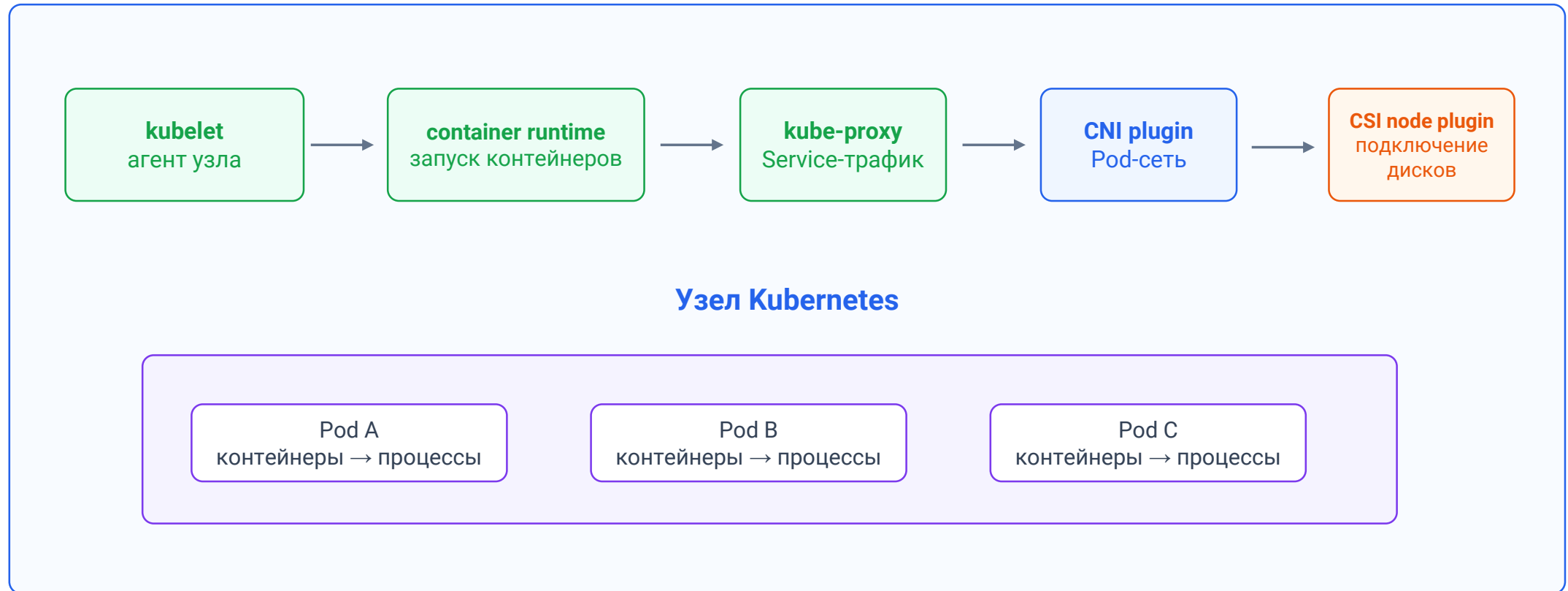
controller-manager

Набор контроллеров: сверяет desired state и real state.

Практический смысл: если API недоступен, нельзя управлять кластером; если etcd потерян – потеряно состояние кластера

Узел Kubernetes: что реально есть на хосте

Node – это VM или физический сервер, на котором Kubernetes запускает Pod



Node характеризуется ресурсами, состоянием, labels/taints и списком Pod. kubelet – главный агент, который связывает control plane и конкретный хост

CRI, CNI, CSI: почему Kubernetes модульный

Kubernetes не обязан сам реализовывать runtime, сеть и storage

CRI

Container Runtime Interface

Как kubelet просит runtime
создать контейнер.

containerd, CRI-O

CNI

Container Network Interface

Как Pod получает сеть и
связность Pod ↔ Pod.

Calico, Cilium, Flannel

CSI

Container Storage Interface

Как Kubernetes подключает
внешнее хранилище.

cloud disk, Ceph, SAN/NAS

Запоминать реализации не обязательно. Важно понять интерфейс: CRI запуск, CNI сеть, CSI диски

Может ли кластер работать без CRI, CNI, CSI?

Без CRI

Практически нет

kubelet не сможет нормально создавать контейнеры через стандартный интерфейс.

Без CNI

Только очень ограниченно

Pod-сеть и связь Pod ↔ Pod между узлами не будут полноценными.

Без CSI

Да, для stateless

Можно жить без динамического persistent storage, но базы и диски станут проблемой.

Для новичков: CRI/CNI/CSI - это не «дополнительные красивые слова», а точки подключения реальных механизмов хоста, сети и хранилища

Почти всё в Kubernetes это объект API

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels:
    app: backend
spec:
  replicas: 3
  template: ...
status:
  availableReplicas: 3
```

apiVersion

какая версия API

kind

тип объекта

metadata

имя, namespace, labels

spec

желаемое состояние

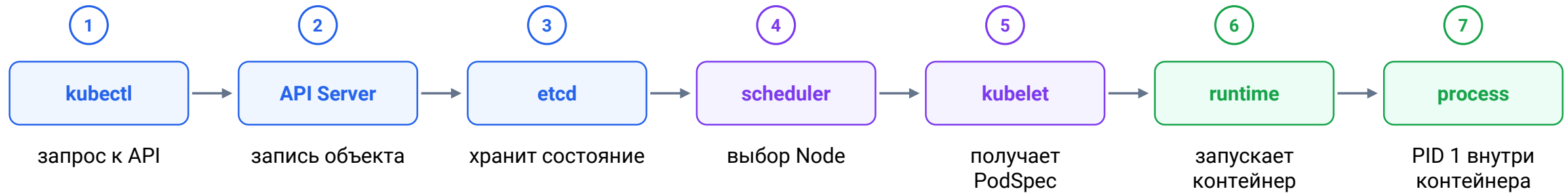
status

текущее состояние

Пользователь обычно задаёт spec. Kubernetes сам обновляет status.

Как Kubernetes запускает Pod

Путь от команды пользователя до процесса внутри контейнера



Эта цепочка помогает понять, почему Deployment/Pod «хранятся» в API, а реально исполняется только процесс на worker node.

Важно

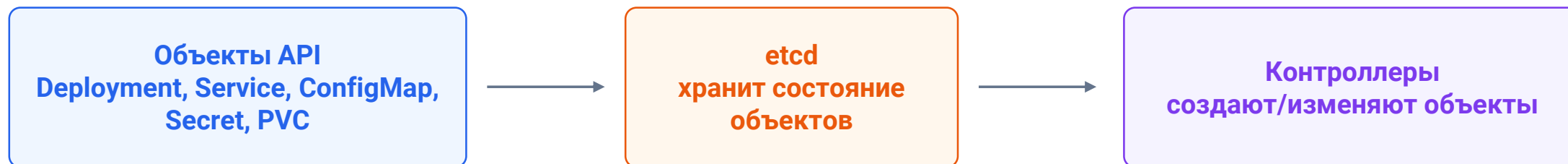
Deployment, Service, ConfigMap – это записи в API. Они не исполняются сами.

Что реально живет

Реально исполняется только процесс внутри контейнера, который находится внутри Pod.

Где хранится и где выполняется объект

Один из самых важных моментов для понимания Kubernetes



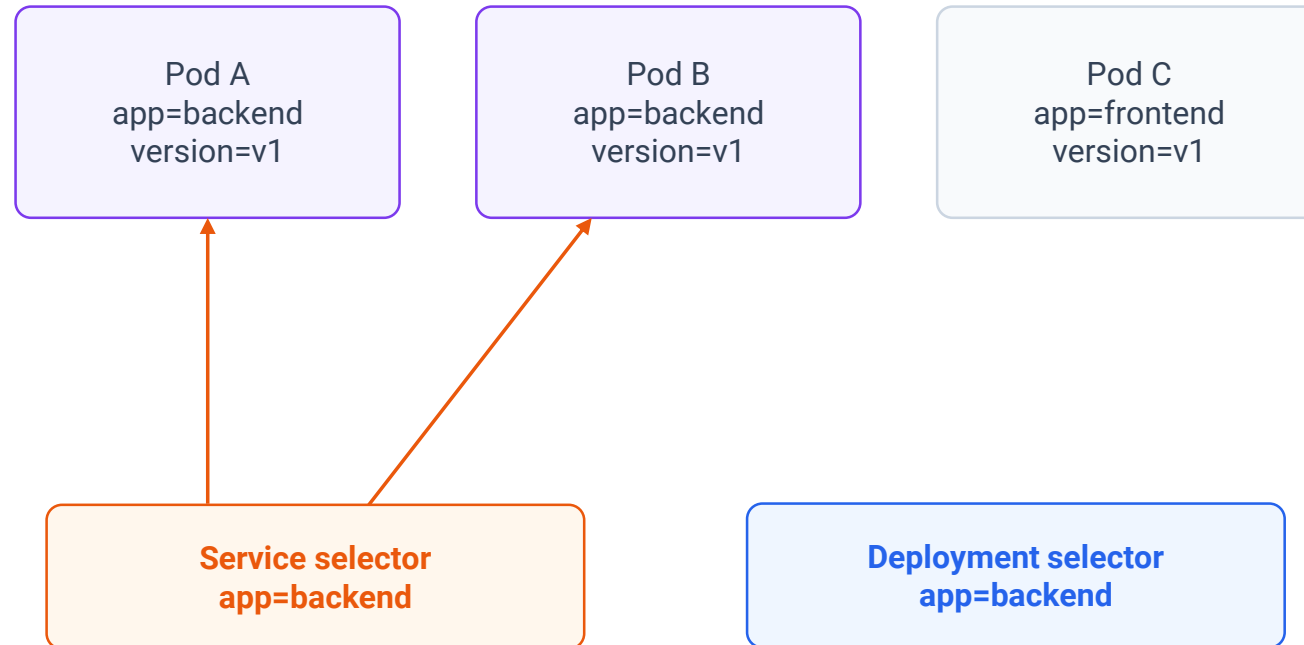
Реально выполняется только процесс внутри контейнера

Pod → Контейнер → Процесс PID 1

Deployment не работает как процесс. Это запись в API. Работают контейнерные процессы в Pod на worker node.

Labels и selectors: клей Kubernetes

Без labels сложно понять Service и Deployment.



Label – метка объекта. **Selector** – правило выбора объектов по меткам. Так Kubernetes связывает Deployment, ReplicaSet, Pod, Service и NetworkPolicy.

Namespaces: логическое разделение объектов

Namespace – это "папка" объектов API: рядом в одном кластере, но с разными именами, правами и квотами

Кластер Kubernetes

namespace: dev

Deployment
backend

Service
backend

Secret
db-pass

ConfigMap
app-cfg

Имена могут повторяться
в разных namespace

namespace: stage

Deployment
backend

Service
backend

Secret
db-pass

ConfigMap
app-cfg

Имена могут повторяться
в разных namespace

namespace: prod

Deployment
backend

Service
backend

Secret
db-pass

ConfigMap
app-cfg

Имена могут повторяться
в разных namespace

cluster-scoped объекты: Node • PersistentVolume • StorageClass • ClusterRole

Важно: Kubernetes Namespace ≠ Linux namespace. Это не полноценная граница безопасности сама по себе.

Обычно namespace связывают с RBAC, ResourceQuota, LimitRange, NetworkPolicy и организацией окружений.

Node: что Kubernetes знает об узле

Scheduler выбирает узел не на глаз, а по данным Node.

capacity

сколько всего CPU/RAM

allocatable

сколько доступно Pod

conditions

Ready, MemoryPressure,
DiskPressure

labels

роль, зона, тип железа

taints

кого не пускать без toleration

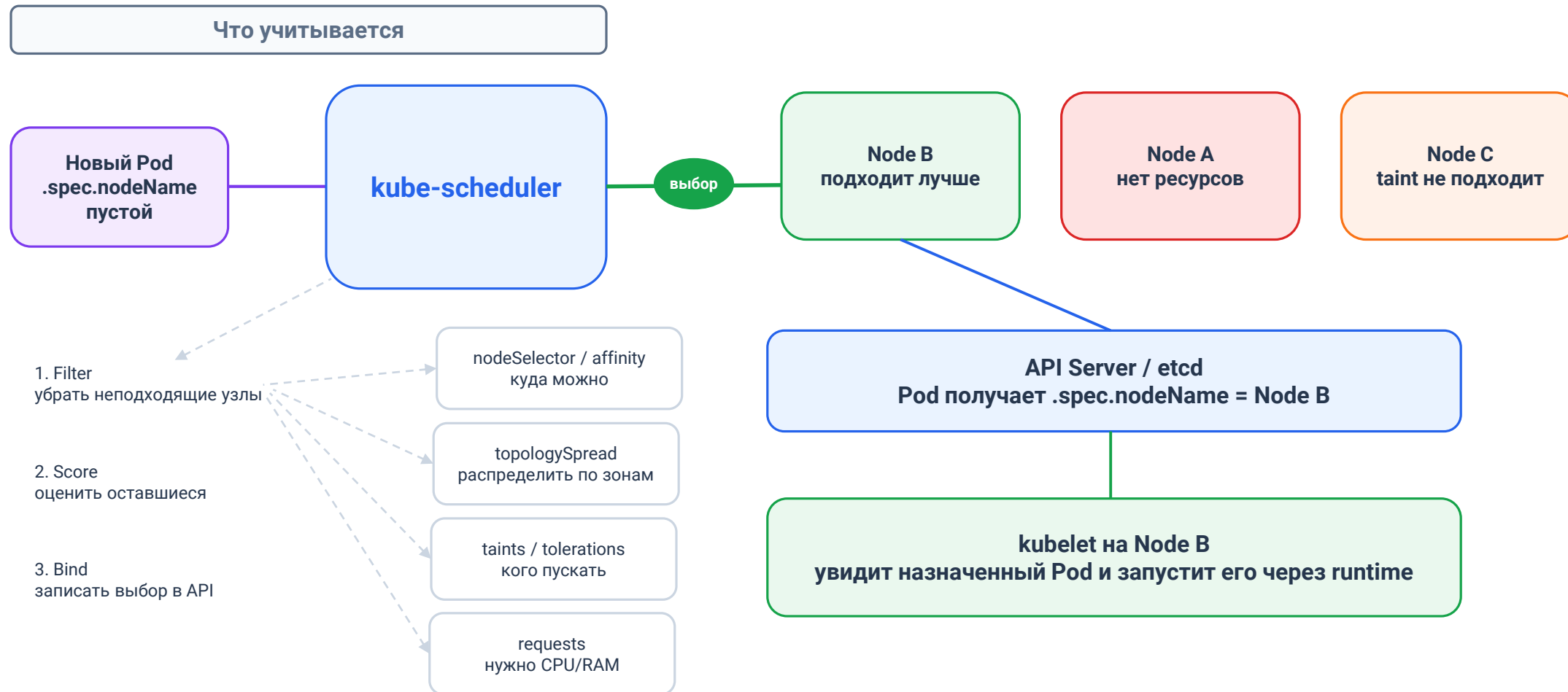
running Pods

что уже размещено

Смысл: Node – это не просто IP-адрес сервера. Для Kubernetes это объект API с ресурсами, состоянием и правилами размещения

Как выбирается Node для Pod

Планировщик не запускает контейнеры: он только назначает новый Pod на подходящий Node.



Важно: scheduler принимает решение один раз для нового Pod. Если узел потом сломался, пересоздание Pod обычно делает контроллер выше уровнем: Deployment/ReplicaSet/StatefulSet.

Pod: минимальная единица запуска

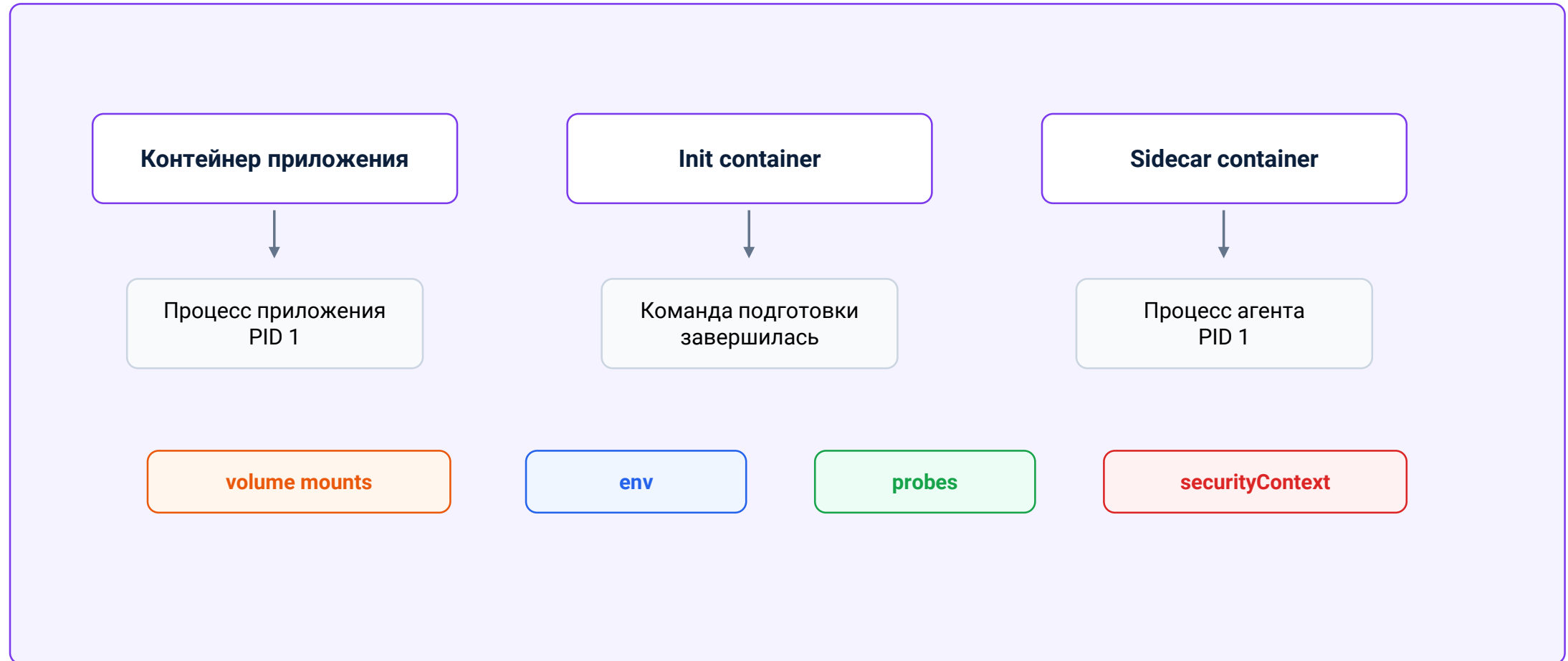
В Kubernetes *обычно* не запускают контейнер напрямую, а запускают Pod.



Pod – это логический хост для одного или нескольких тесно связанных контейнеров. Контейнеры внутри Pod запускаются на одном узле и разделяют сетевой контекст.

Что внутри Pod

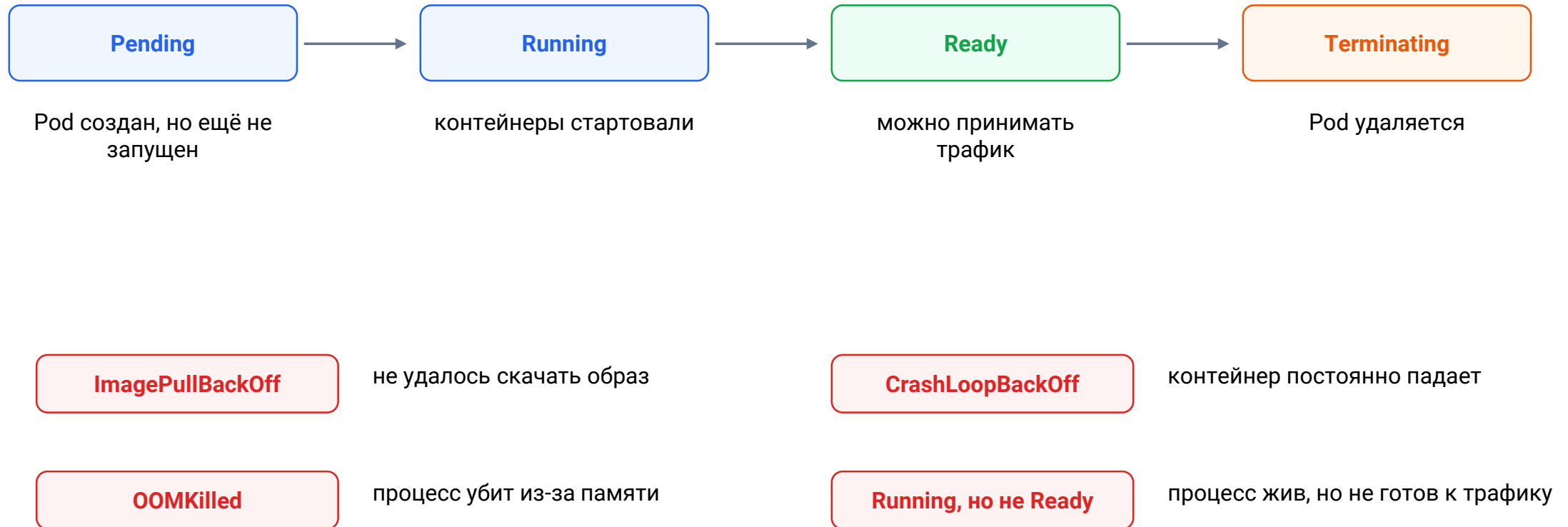
Иерархия: Pod → контейнер → процесс.



Pod сам по себе не приложение, а упаковка для контейнеров и параметров их запуска.

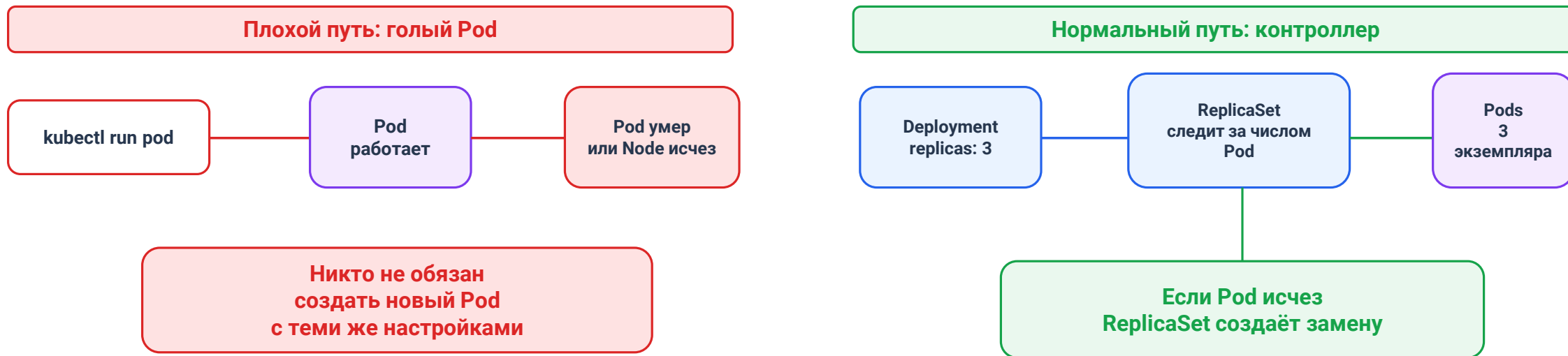
Жизненный цикл Pod и частые состояния

Эти слова быстро встретятся в kubectl.



Почему не надо запускать Pod вручную

Pod временный: Kubernetes обычно управляет приложением через контроллеры, а не через одиночный Pod.



Запомнить

Для обычного stateless-приложения базовый объект управления – Deployment.
Голый Pod полезен для теста/отладки, но не для устойчивого запуска приложения.

От Deployment до работающего Pod

Кто создает задачу на Pod, как Pod появляется на Node и откуда берутся probes/resources.



PodSpec приходит из Deployment.template

```
template:
  spec:
    containers:
    - image: backend:v1
      resources:
        requests: {cpu: 100m, memory: 128Mi}
        limits: {memory: 256Mi}
      startupProbe: ...
      readinessProbe: ...
      livenessProbe: ...
```

Кто использует поля?

requests
scheduler
для выбора Node

limits
kernel / cgroups
для ограничения
ресурсов

probes
kubelet
для проверок контейнера

livenessProbe

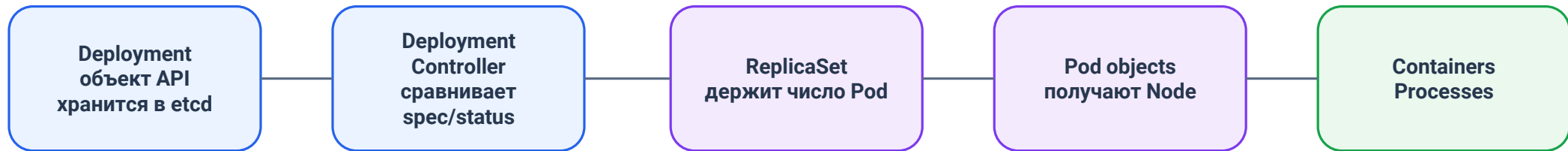
startupProbe

readinessProbe

Probes не приходят снаружи во время работы. Они описаны в PodSpec, а kubelet на узле регулярно выполняет эти проверки и обновляет состояние Pod/Container.

Deployment: объект, контроллер и реальные процессы

Deployment не выполняется как программа. Это объект API, по которому контроллер создаёт ReplicaSet и Pod.



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 3           # сколько Pod нужно
  selector:             # какие Pod принадлежат ReplicaSet
    matchLabels:
      app: backend
  template:             # шаблон будущих Pod
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: backend:v1
  strategy:
    type: RollingUpdate
```

На что смотреть в Deployment

Replicas – сколько экземпляров хотим

selector  labels – как Deployment находит свои Pod

template – какие Pod будут созданы

image – какую версию приложения запустить

strategy – как обновлять без простоя

Почему здесь несколько объектов: Deployment отвечает за rollout, ReplicaSet за количество Pod, Pod за запуск контейнеров, а реальные процессы живут только внутри контейнеров на Node.

Deployment manifest: на что смотреть

Пример с минимальными важными полями.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 3                # сколько Pod хотим
  selector:
    matchLabels:
      app: backend          # какие Pod принадлежат
  strategy:
    type: RollingUpdate
  template:                  # шаблон будущих Pod
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: registry/backend:v1
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
```

replicas

сколько экземпляров приложения

selector

связь Deployment ↔ Pod

template

как будут выглядеть новые Pod

image

какую версию запускать

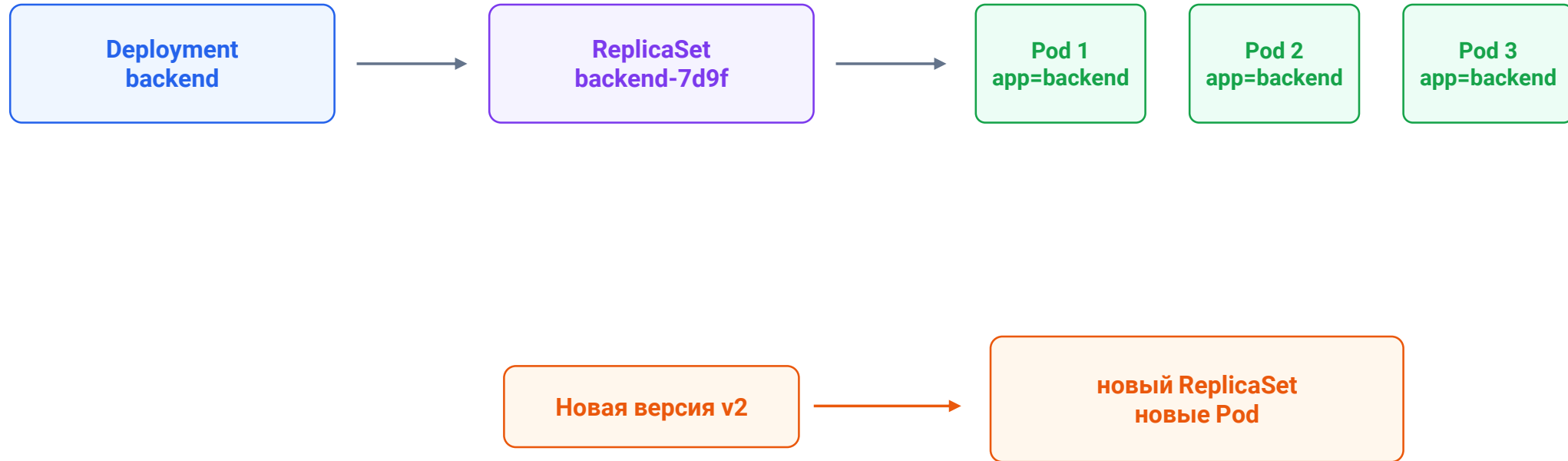
resources

что учесть при scheduling

Самое частое место ошибки: selector и template.labels должны совпадать по смыслу.

Deployment → ReplicaSet → Pod

Связь объектов становится понятной через ownership.



При rollout Deployment создаёт новый ReplicaSet и постепенно переводит трафик на новые Ready Pod через Service.

Другие workload-объекты

Deployment база, но не единственный способ запускать workload.

Deployment

stateless web/API, rolling update

StatefulSet

стабильная идентичность Pod и диски

DaemonSet

один Pod на каждом или выбранных узлах

Job

одноразовая задача до завершения

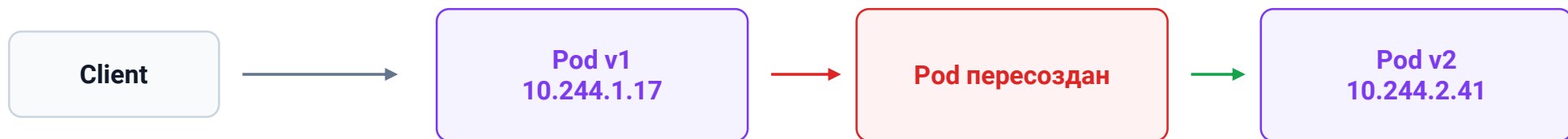
CronJob

задача по расписанию

Для первого знакомства достаточно глубоко понять Deployment, Pod и Service. Остальные объекты легче изучать позже по задачам.

Проблема сети: Pod IP нестабилен

Если Pod пересоздаётся, его IP может измениться.

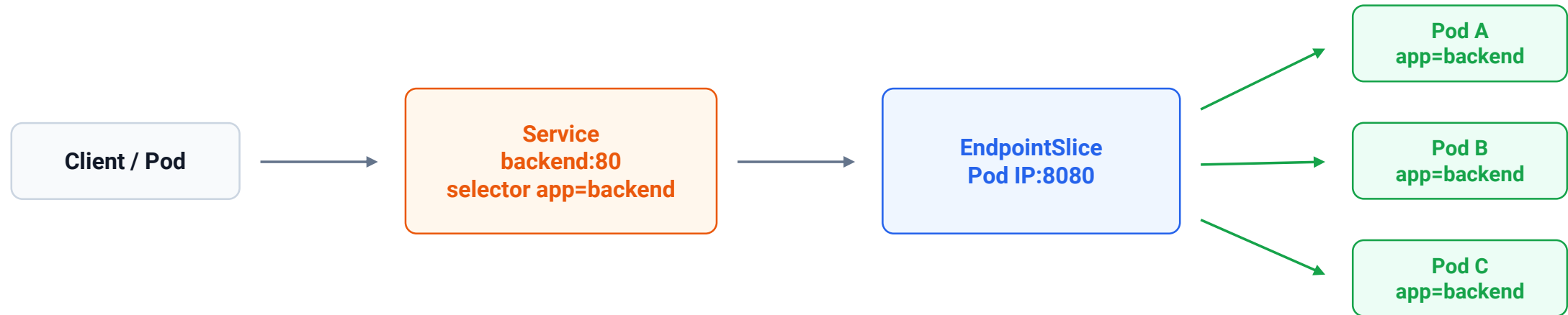


Нельзя строить архитектуру на прямом обращении к Pod IP.

Нужна стабильная точка доступа, которая сама найдёт актуальные Pod

Service: стабильный L4-доступ к Pod

Service выбирает Pod по selector и даёт стабильный адрес.



Service обычно работает на L4: IP/port/protocol. HTTP path и host это уже зона Ingress.

Типы Service

Один объект Service может решать разные задачи доступа.

ClusterIP

доступ только внутри кластера
базовый вариант

NodePort

порт на каждом узле
часто для тестов

LoadBalancer

внешний балансировщик
часто в облаке

ExternalName

DNS-алиас на внешнее имя
редкий кейс

Надо сначала понять ClusterIP. NodePort и LoadBalancer способы вывести Service наружу.

Service это не Ingress

Частая путаница: оба про трафик, но на разных уровнях.

Service

- стабильный IP/DNS внутри кластера
- работа с port/protocol
- выбор Pod через selector
- обычно L4

Ingress

- HTTP/HTTPS вход снаружи
- host/path rules
- TLS termination
- направляет в Service

Запомни: Service стабилизирует доступ к Pod, Ingress публикует HTTP/HTTPS правила наружу.

Ingress и Ingress Controller

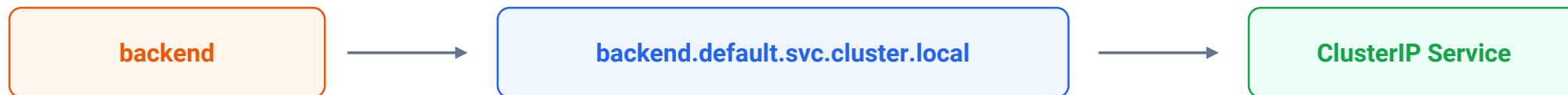
Ingress – это правила. Ingress Controller – компонент, который эти правила реализует.



```
rules:
- host: app.example.ru
  http:
    paths:
    - path: /api
      backend:
        service:
          name: backend
          port: 8080
```


DNS внутри Kubernetes

Приложения обычно обращаются к Service name, а не к Pod IP.



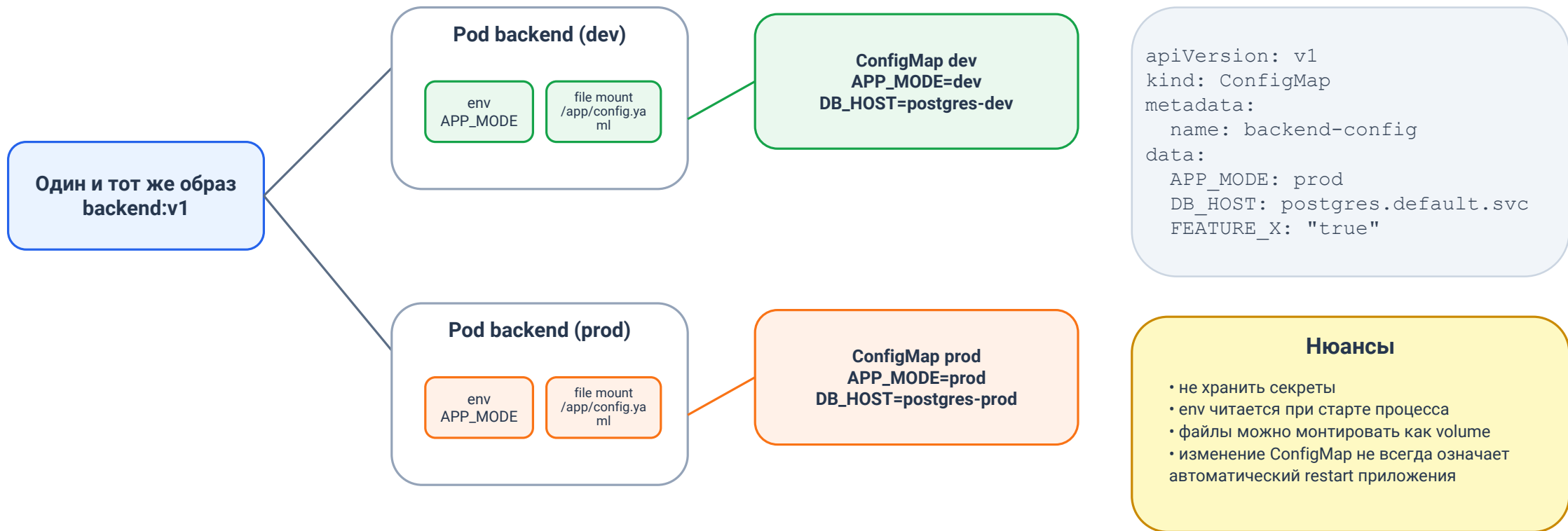
Примеры имен:

```
backend  
backend.default  
backend.default.svc  
backend.default.svc.cluster.local
```

DNS снимает необходимость знать IP конкретного Pod. Это делает приложения переносимыми внутри кластера.

ConfigMap: настройки без пересборки образа

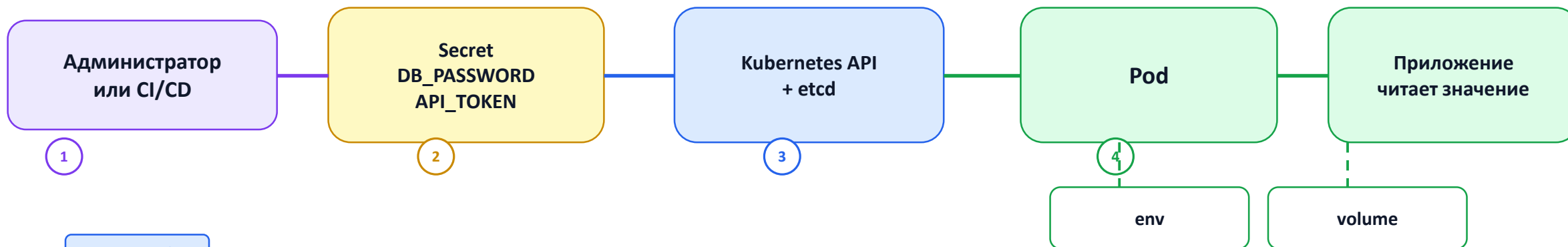
Один образ приложения можно запускать в разных окружениях, меняя внешнюю конфигурацию



ConfigMap это не настройка Deployment, а отдельный объект API. Pod получает значения из него через свой PodSpec: env, envFrom или volume mount.

Secret: чувствительные значения

Secret удобен, но это не магическая защита от утечек.



пример манифеста

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:
  DB_PASSWORD: <значение>
---
containers:
- name: backend
  envFrom:
  - secretRef:
    name: db-secret
```

Что обязательно объяснить новичкам

RBAC

кто может читать Secret через API

encryption at rest

шифровать Secret в etcd

не хранить в Git

использовать sealed/external secrets или CI/CD

rotation

периодически менять значения и токены

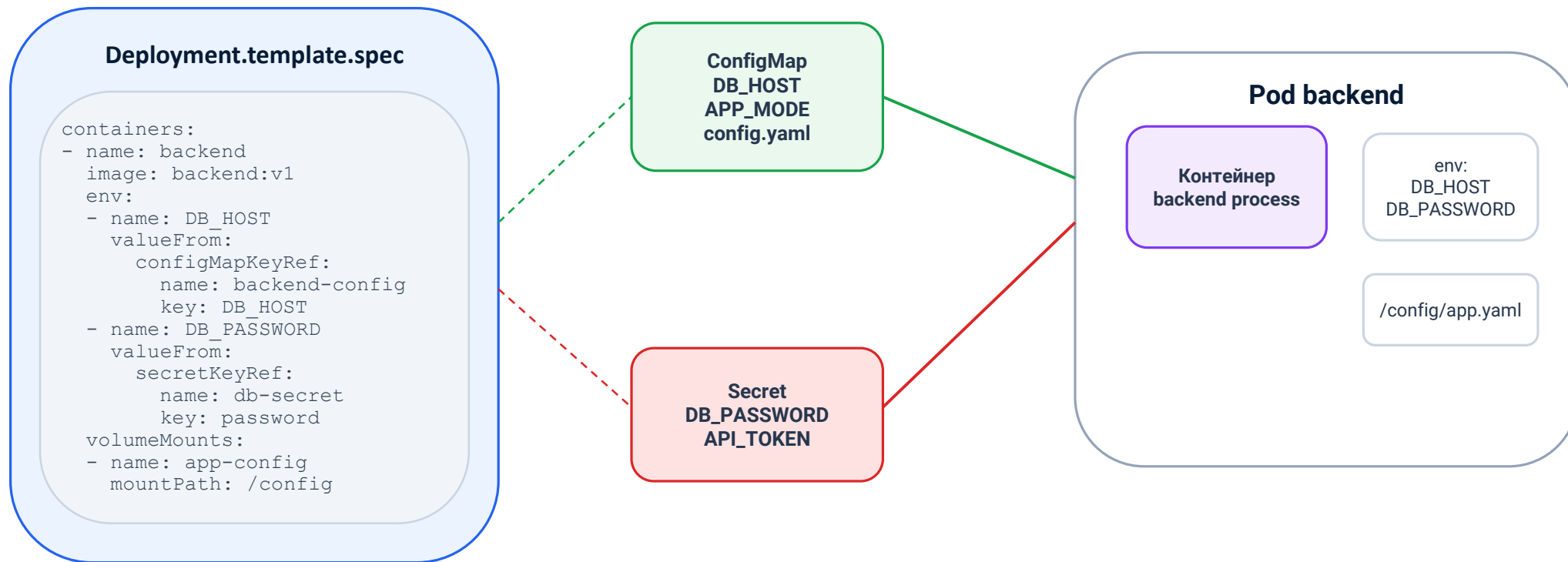
логирование

не печатать env/секреты в logs и errors

Ключевая мысль: Secret уменьшает хаос с паролями, но безопасность появляется только вместе с RBAC, шифрованием etcd, аудитом, ротацией и дисциплиной хранения.

Как Pod получает ConfigMap и Secret

Связь задаётся внутри Pod template: значения попадают в контейнер как env или как файлы.

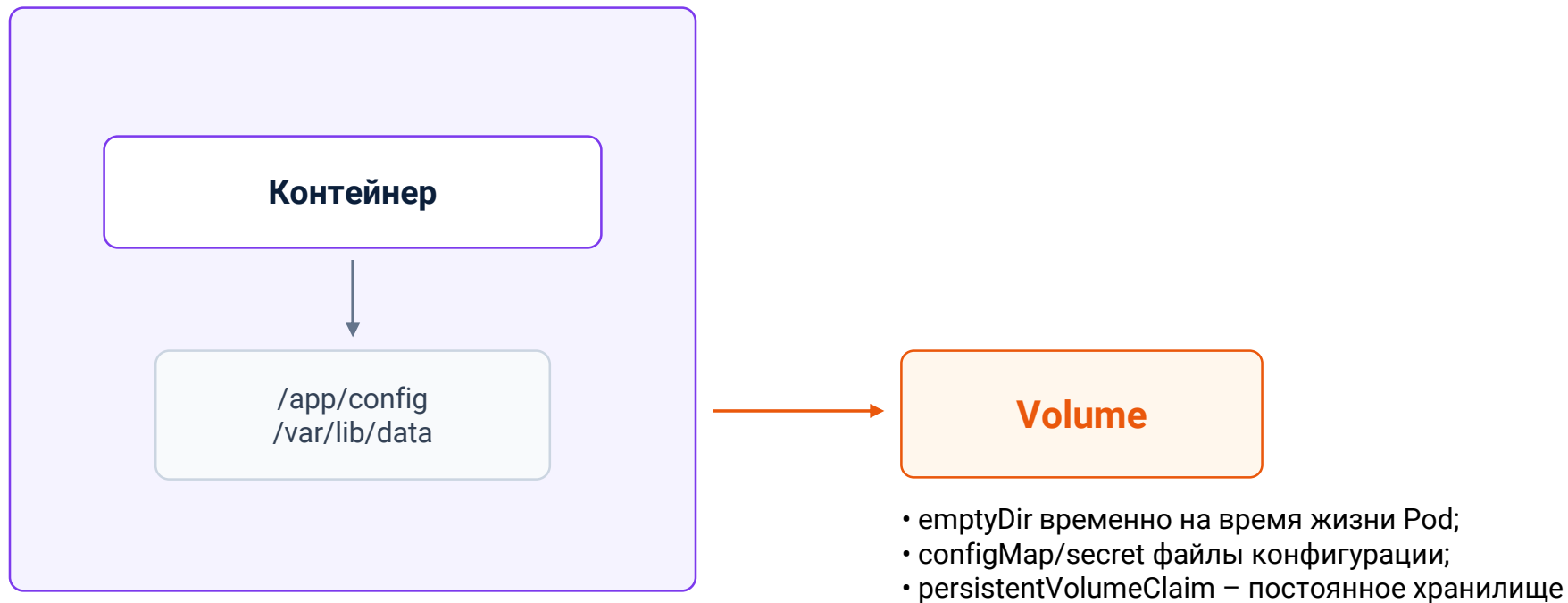


ConfigMap/Secret не исполняются и не подключаются к Deployment напрямую. Deployment только содержит шаблон Pod, а уже созданный Pod получает env/файлы из этих объектов.

Важно для ИБ: Secret в Kubernetes – это объект API с base64-данными; защищать его надо через RBAC, encryption at rest, аудит и запрет утечек в логи/Git.

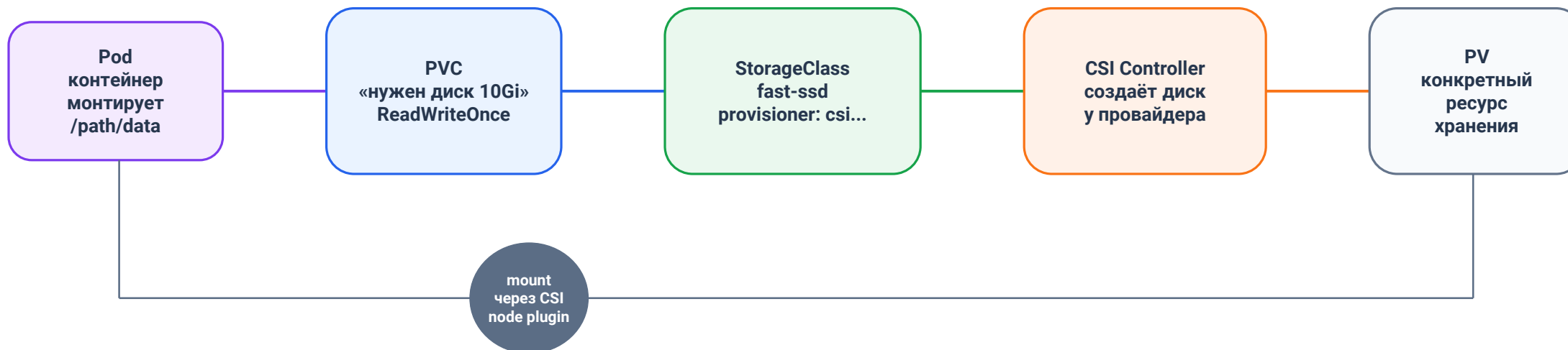
Volume: файловая система для контейнера

Pod временный, но приложению часто нужны файлы.



PVC, PV, StorageClass: цепочка запроса диска

Хранилище лучше понимать как заявку от приложения и реализацию со стороны платформы.



Что можно выбирать

- storageClassName: fast-ssd / slow-hdd
- accessModes: RWO / RWX
- resources.requests.storage: 10Gi
- reclaimPolicy: Retain / Delete

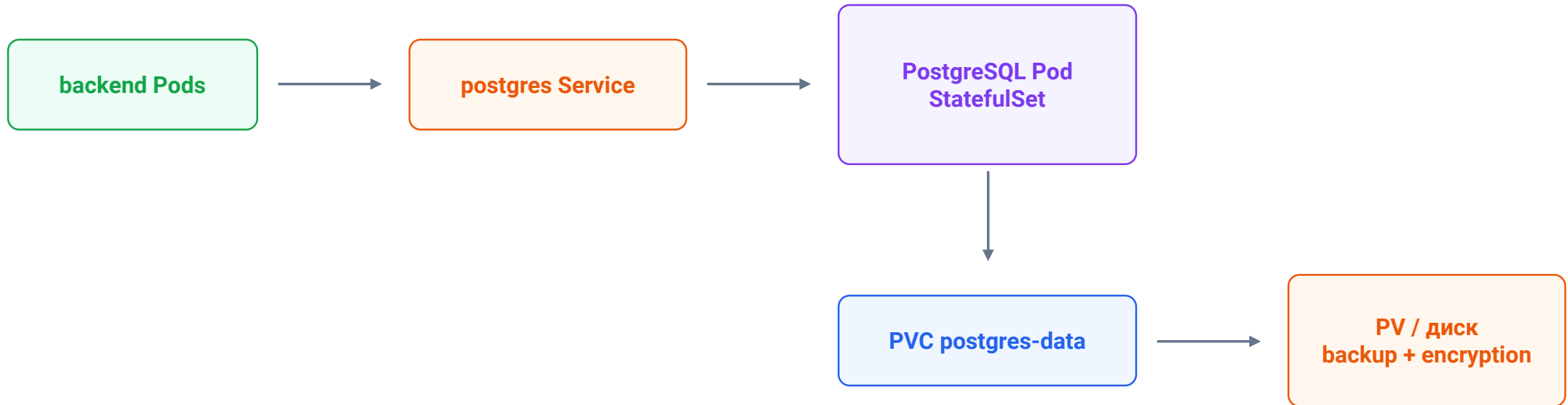
Реальный кейс

PostgreSQL Pod получает PVC "pgdata". Платформа через StorageClass выдаёт диск.
При пересоздании Pod новый экземпляр снова монтирует тот же volume, а не теряет данные.

Без CSI можно использовать простые локальные/временные volume, но нормальная динамическая выдача persistent-дисков обычно требует CSI-драйвер конкретной системы хранения.

Реальный storage-кейс: PostgreSQL

Здесь storage становится не абстракцией, а требованием приложения.



Важные вопросы: access mode, reclaim policy, backup/restore, шифрование, snapshot, что будет при удалении PVC.

Минимум ИБ: карта рисков Kubernetes

API / RBAC

кто может менять кластер

Pod privileges

что может процесс внутри
контейнера

NetworkPolicy

куда Pod может ходить

Secrets

как не утекут токены и пароли

Images

что именно запускается

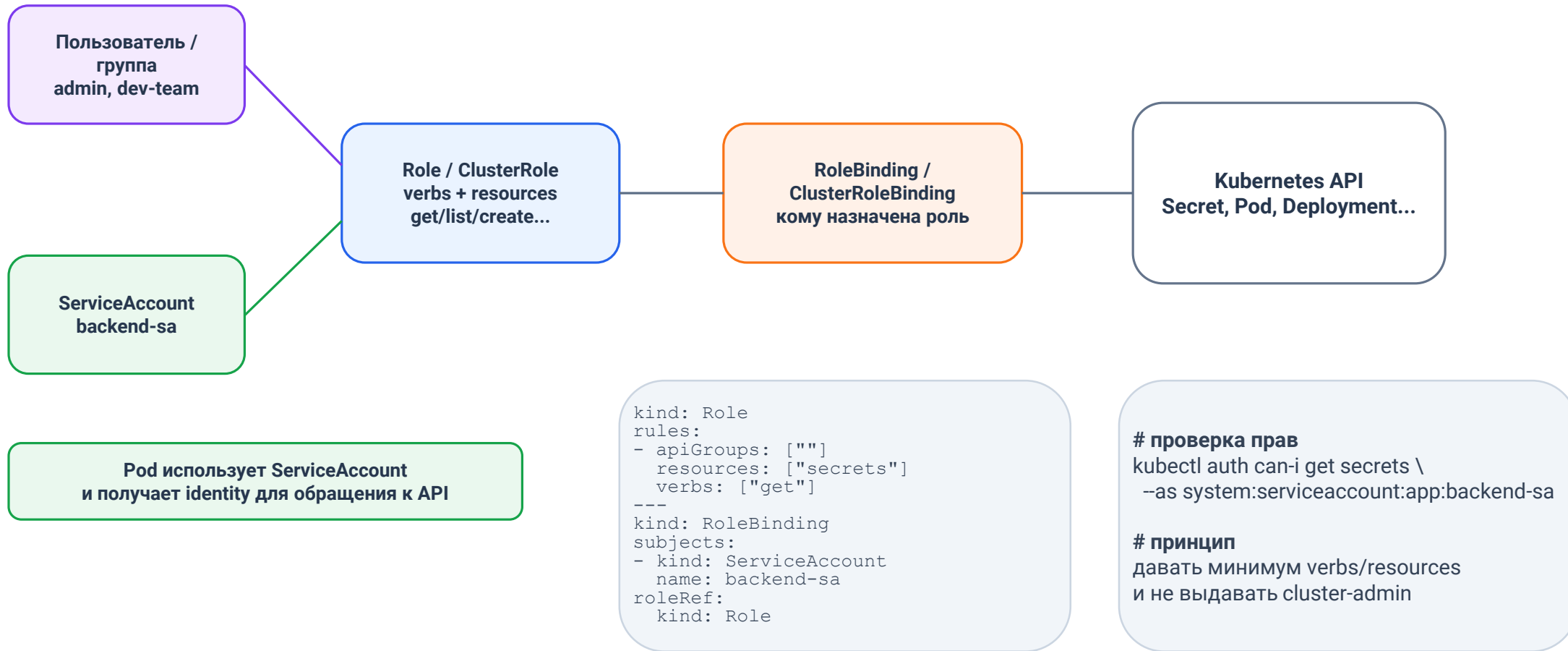
Storage

кто видит данные и backup

Минимальный принцип: least privilege для людей, сервисов и Pod; ограниченные контейнеры; контролируемая сеть; проверенные образы.

RBAC и ServiceAccount: кто имеет право на действие

В Kubernetes доступы есть у людей, автоматизаций и у самих workload через ServiceAccount.



Контрольные вопросы: может ли этот Pod читать Secret? может ли пользователь создавать privileged Pod? может ли CI/CD менять RBAC?
Ответ должен быть проверяемым, а не "кажется, не может".

Pod Security: как ограничивать контейнер

Ограничения задаются в securityContext и enforced через Pod Security Admission / policy engine.

```
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    spec:
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
      - name: backend
        image: backend:v1
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities:
            drop: ["ALL"]
        resources:
          limits:
            memory: 256Mi
```

Что делает каждая настройка

runAsNonRoot
процесс не стартует от root

allowPrivilegeEscalation: false
запрет повышения привилегий

readOnlyRootFilesystem
root filesystem только для чтения

capabilities.drop: ALL
убрать лишние Linux capabilities

seccompProfile: RuntimeDefault
ограничить системные вызовы

resources.limits
снизить риск OOM/DoS на узел

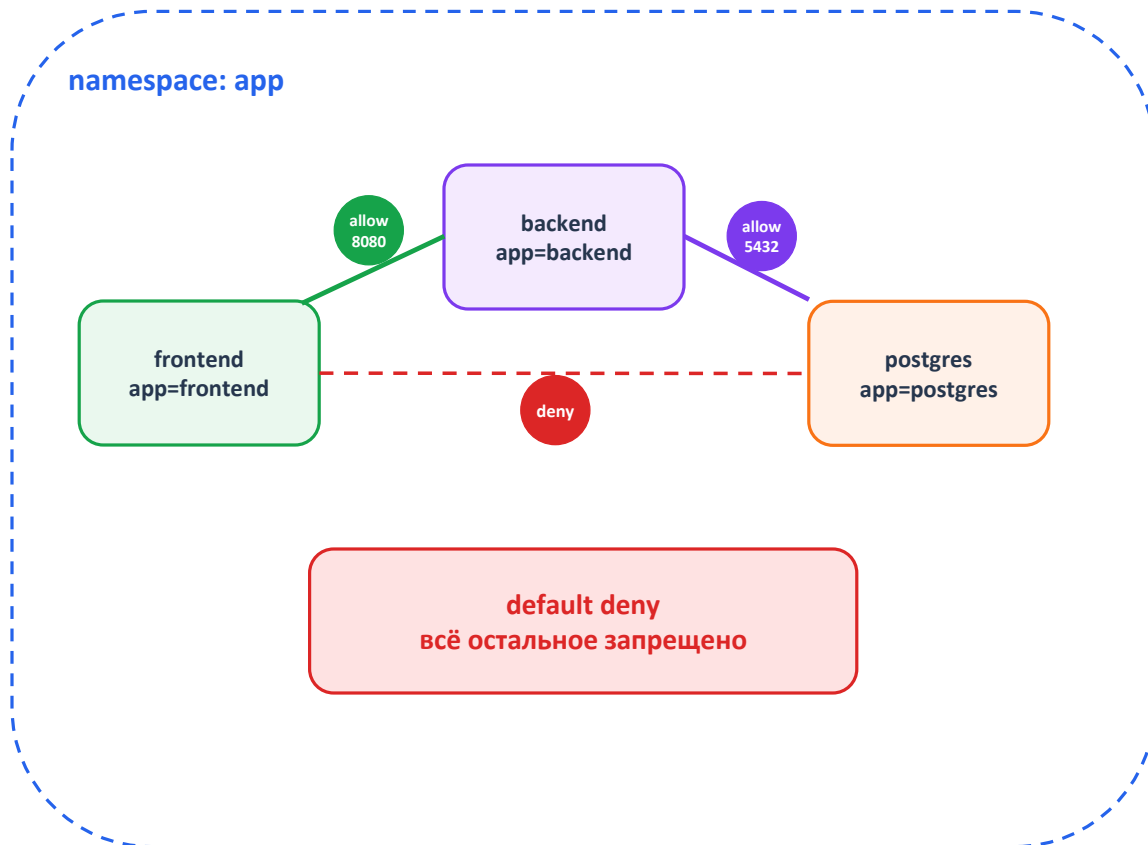
Где применять

- Pod Security Standards: baseline/restricted
- Admission Controller / policy engine
- CI-проверка манифестов до деплоя

Это не оптимизация, а защита от расширения компрометации: приложение может быть взломано, но контейнер не должен легко превратиться в доступ к Node.

NetworkPolicy: управляем lateral movement

Без политики сеть часто оказывается слишком свободной.



```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-ingress
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes: [Ingress]
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: frontend
    ports:
      - port: 8080
```

Как читать манифест

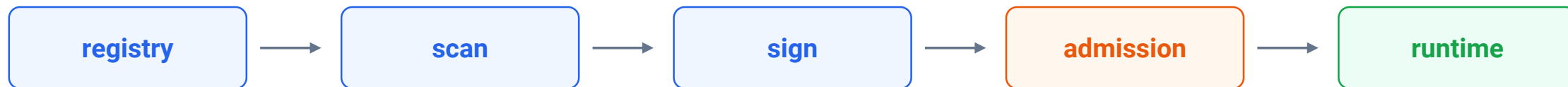
podSelector = к кому применяется политика
from = кто может подключаться
ports = на какие порты разрешён доступ

Для backend → postgres обычно нужна отдельная policy: либо egress от backend, либо ingress на postgres.

Важно: NetworkPolicy работает только при CNI-плагине с поддержкой сетевых политик. Если CNI не enforce-ит политики, манифест может существовать, но трафик фактически не фильтруется.

Image supply chain и admission

Кластер должен запускать не что угодно, а проверенные образы

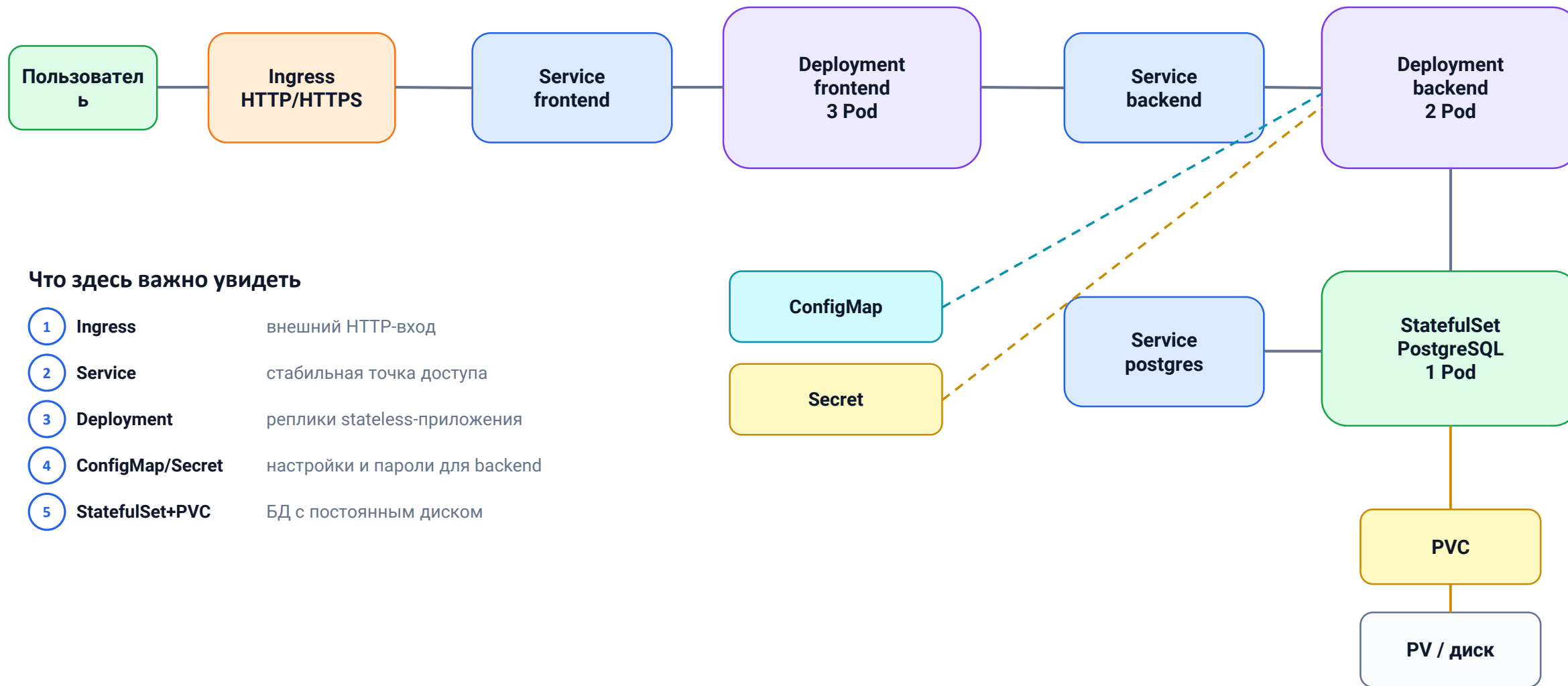


- не использовать :latest для production
- закреплять digest, где это нужно
- сканировать образы и базовые слои
- ограничивать registries через policy
- не хранить секреты в образе

Admission policy – точка, где можно запретить небезопасный объект до запуска.

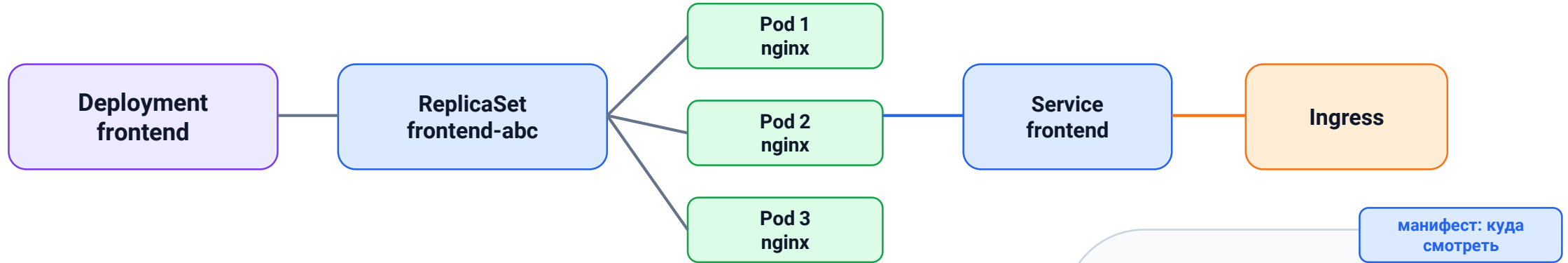
Лаба: простой пример "не простой" архитектуры

Собираем не отдельный nginx, а минимальное приложение с реальными связями объектов Kubernetes



Лаба: frontend Deployment

Deployment это не процесс, а объект API, который через ReplicaSet поддерживает нужное число Pod



Если Pod умер, ReplicaSet создаст новый. Если меняется template в Deployment, появится новый ReplicaSet и начнётся rollout.

Пять ключевых мест

replicas	сколько Pod хотим
selector	какие Pod принадлежат
template.labels	метки будущих Pod
image	что запускаем
readinessProbe	когда можно давать трафик

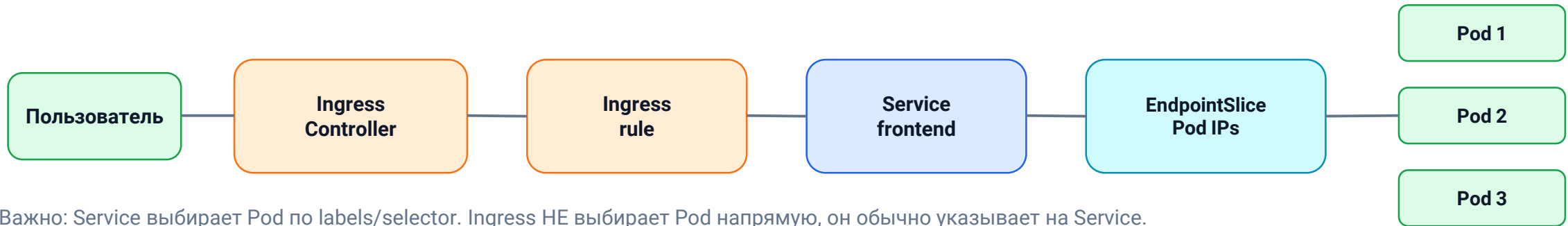
```
kubectl apply -f frontend.yaml
kubectl get deploy,rs,pod -o wide
kubectl describe deploy frontend
```

манифест: куда смотреть

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
          readinessProbe: ...
          resources: ...
```

Лаба: Service + Ingress

Service даёт стабильный L4-доступ к Pod, Ingress принимает HTTP/HTTPS снаружи и направляет его в Service.



Важно: Service выбирает Pod по labels/selector. Ingress НЕ выбирает Pod напрямую, он обычно указывает на Service.

Service = L4

```
kind: Service
metadata:
  name: frontend
spec:
  type: ClusterIP
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
```

Ingress = L7 HTTP

```
kind: Ingress
metadata:
  name: frontend
spec:
  rules:
    - host: app.local
      http:
        paths:
          - path: /
            backend:
              service:
                name: frontend
                port:
                  number: 80
```

Как проверить

```
kubectl get svc
```

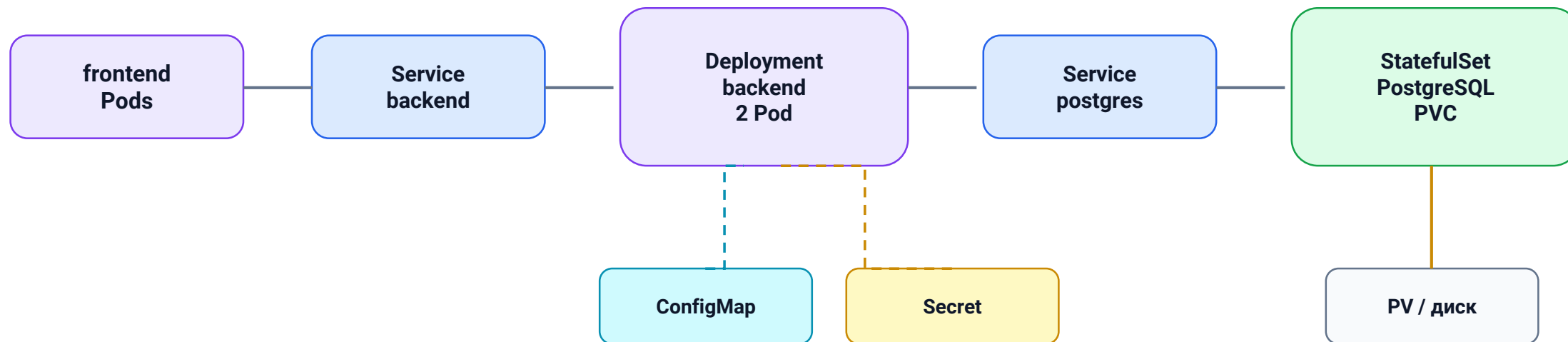
```
kubectl get endpointslice
```

```
kubectl describe ingress
```

```
curl http://app.local
```

Лаба: backend + ConfigMap/Secret + PostgreSQL PVC

Показываем реальные зависимости: код отдельно, настройки отдельно, секреты отдельно, данные отдельно



backend получает config/secret

```
env:
- name: DB_HOST
  valueFrom:
    configMapKeyRef:
      name: backend-config
      key: DB_HOST
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: DB_PASSWORD
```

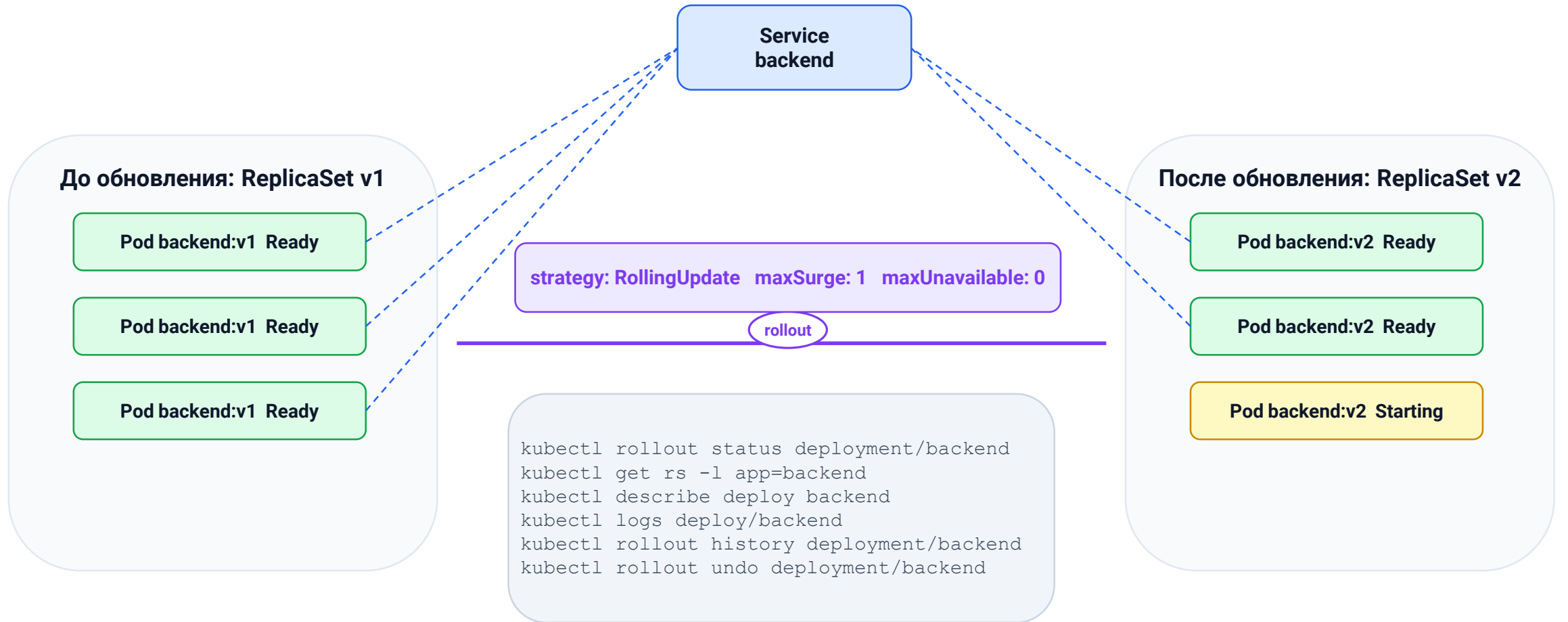
postgres получает постоянный диск

```
volumeMounts:
- name: pgdata
  mountPath: /var/lib/postgresql/data
volumes:
- name: pgdata
  persistentVolumeClaim:
    claimName: postgres-pvc
```

Ключевой порядок мысли: Service соединяет приложения, ConfigMap/Secret настраивают backend, PVC сохраняет состояние PostgreSQL между пересозданиями Pod.

RollingUpdate на примере backend API

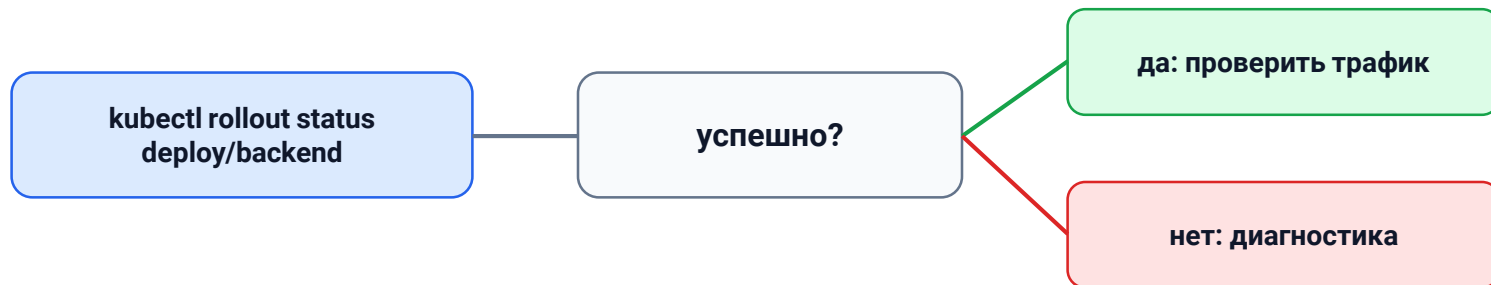
Обновление – это управляемая замена Pod: Service должен видеть только готовые экземпляры приложения



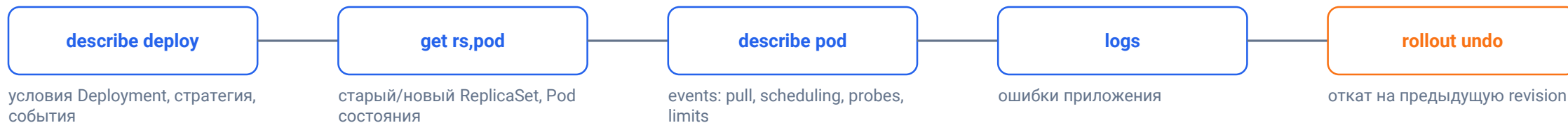
- 1 Deployment меняет template: image v1 → v2
- 2 Создаётся новый ReplicaSet
- 3 Новые Pod проходят readinessProbe
- 4 Service даёт трафик только Ready Pod
- 5 Старые Pod постепенно удаляются

Rollout status и rollback

Обновление надо не только запустить, но и уметь наблюдать, диагностировать и откатывать.



Маршрут диагностики

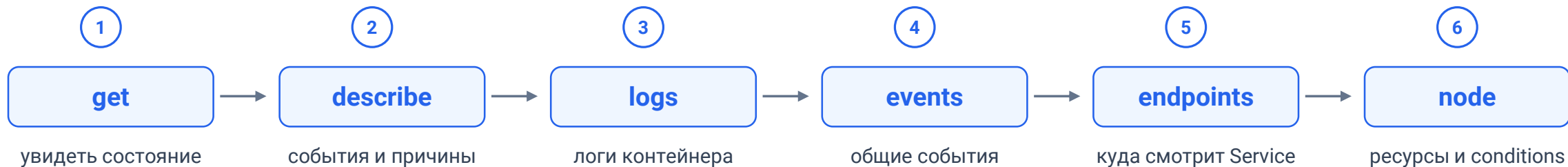


```
kubectl rollout status deployment/backend
kubectl rollout history deployment/backend
kubectl describe deployment backend
kubectl get rs,pod -l app=backend -o wide
kubectl describe pod <pod>
kubectl logs deploy/backend --previous
kubectl rollout undo deployment/backend
```

Важная оговорка

Rollback возвращает манифест и Pods к предыдущей версии, но не откатывает автоматически миграции базы данных, внешние API и уже записанные данные.

Базовый troubleshooting: маршрут диагностики



```
kubectl get pod -o wide
kubectl describe pod <pod>
kubectl logs <pod> -c <container>
kubectl get events --sort-by=.lastTimestamp
kubectl get svc,endpointslice
kubectl describe node <node>
```

Итоговая карта Kubernetes

Карта, по которой можно повторить всю презентацию

Архитектура

control plane, node, kubelet,
runtime

Workloads

Pod, Deployment, ReplicaSet,
StatefulSet

Сеть

Service, EndpointSlice, Ingress,
DNS, CNI

Storage

Volume, PVC, PV, StorageClass,
CSI

Config

ConfigMap, Secret, env, mount

ИБ

RBAC, Pod Security,
NetworkPolicy, images

Операции

rollout, rollback, logs, events

Платформа

HA, GitOps, observability, policy

Kubernetes легче изучать как карту взаимосвязей, а не как список YAML-файлов.

Что изучать дальше и первоисточники

Закрепление и направление самостоятельного изучения

- kubectl и YAML: get/describe/logs/apply/delete
- Pods, Deployments, Services, Ingress
- ConfigMap, Secret, PVC/StorageClass
- RBAC, ServiceAccount, Pod Security, NetworkPolicy
- Helm или Kustomize для packaging
- Observability: logs, metrics, traces, events
- GitOps: Argo CD / Flux
- CNI/CSI и архитектура production-кластера

Официальные источники:

<https://kubernetes.io/docs/concepts/overview/components/>

<https://kubernetes.io/docs/concepts/workloads/pods/>

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

<https://kubernetes.io/docs/concepts/services-networking/service/>

<https://kubernetes.io/docs/concepts/services-networking/ingress/>

<https://kubernetes.io/docs/concepts/storage/persistent-volumes/>

<https://kubernetes.io/docs/concepts/storage/storage-classes/>

Финальная мысль: Kubernetes – это не один инструмент, а слой управления жизненным циклом приложения поверх хостов, сети и хранилищ.